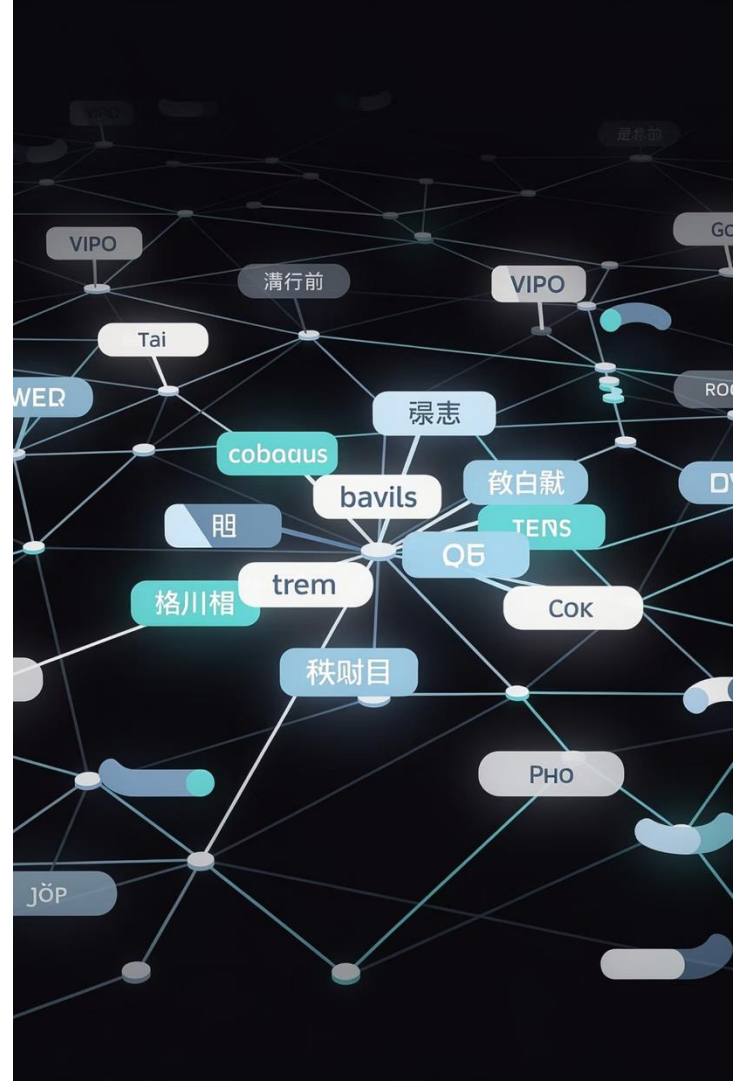


Análisis de Riqueza Léxica en NLP

TTR, MATTR y por qué tu sistema RAG los necesita para no perder la puntería semántica

PROCESAMIENTO DE LENGUAJE NATURAL

NIVEL UNIVERSITARIO / POSGRADO



La Alacena de las Palabras

Tokens vs. Types: dos formas de contar el vocabulario

TOKEN

Cada palabra tal como aparece escrita en el texto, **incluyendo repeticiones**. Es la cuenta bruta, la cantidad total de elementos en la secuencia.

 «El decreto aprueba el presupuesto» → **5 tokens**
(«el» aparece dos veces y cuenta dos veces)

TYPE

Cada **forma única** de palabra que aparece en el vocabulario del texto, independientemente de cuántas veces se repita. Es el inventario real de ingredientes lingüísticos distintos.

 «El decreto aprueba el presupuesto» → **4 types**
{el, decreto, aprueba, presupuesto}

Esta distinción fundamental es la base de todos los índices de riqueza léxica. Cuantos más types únicos tengamos respecto al total de tokens, mayor es la diversidad vocabular del texto analizado.

La Fórmula del TTR

Type-Token Ratio: midiendo la densidad léxica

Definición Formal

$$TTR = \frac{\text{Types (formas únicas)}}{\text{Tokens (total de palabras)}}$$

El resultado oscila siempre entre **0.0** y **1.0**. Cuanto más cercano a 1, mayor es la variedad léxica del texto.

Interpretación del valor

TTR = 1.0 — Variedad total

Cada palabra en el texto es única. Máxima diversidad léxica posible.

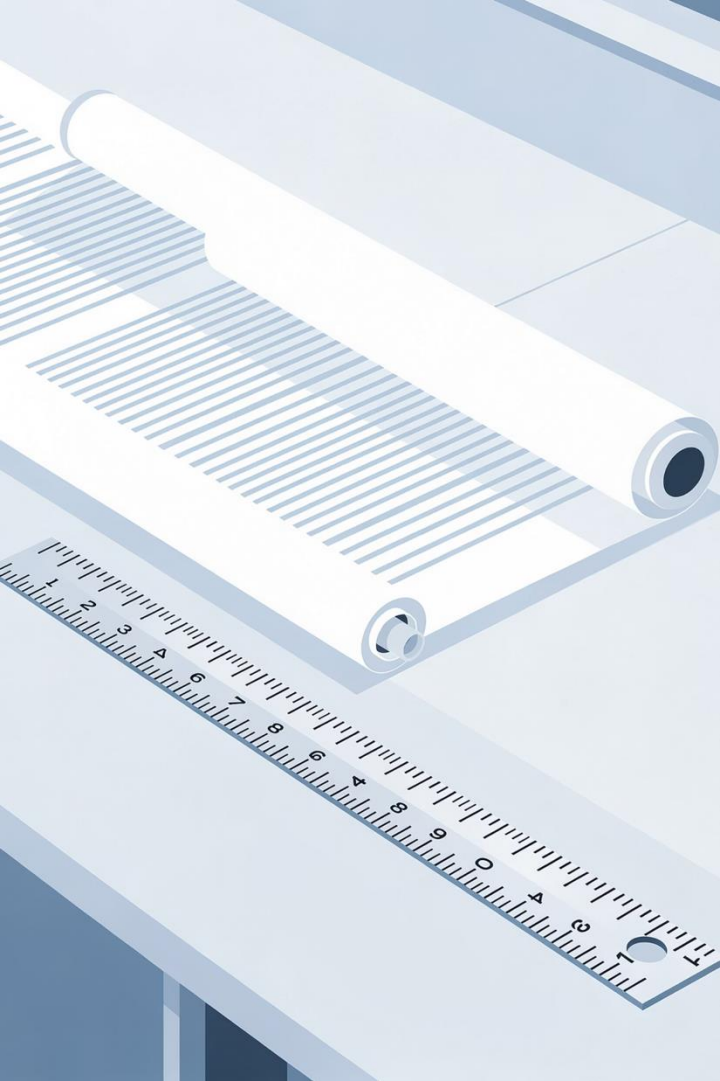
TTR ≈ 0.5 — Equilibrio moderado

Mezcla razonable de términos nuevos y reiterados. Textos narrativos típicos.

TTR ≈ 0.2 — Repetición masiva

El texto recicla constantemente las mismas palabras. Señal de baja variedad.

📄 Ejemplo rápido: «El decreto aprueba el presupuesto» → $TTR = 4/5 = \mathbf{0.80}$ (variedad alta para esta frase corta)

An illustration on the left side of the page shows a rolled-up document with a ruler placed next to it. The ruler has markings in centimeters and millimeters. The document is partially unrolled, showing a grid-like pattern of lines.

El Sesgo de Longitud

La Ley de Herdan: por qué el TTR simple es injusto con los textos largos

El vocabulario humano es **finito**. A medida que un texto crece, es estadísticamente inevitable que las palabras comiencen a repetirse. Un tweet de 10 palabras y una ley de 100 páginas no pueden compararse directamente usando TTR puro.

1

Tweet (50 palabras)

TTR ≈ 0.85

Vocabulario aparentemente muy rico

2

Artículo (500 palabras)

TTR ≈ 0.60

El ratio baja por acumulación natural

3

Ley / PDF (50.000 palabras)

TTR ≈ 0.15

Caída estadística, no real pobreza léxica



Conclusión clave: Comparar TTR entre textos de diferente longitud produce resultados engañosos. Necesitamos una métrica que neutralice este efecto. Aquí entra el **MATTR**.

La Solución: MATTR

Moving-Average Type-Token Ratio — La Cámara Deslizante



¿Cómo funciona?

En lugar de calcular el TTR sobre todo el texto de una sola vez, MATTR utiliza una **ventana de tamaño fijo** (p. ej. 50 tokens) que se deslaza por el texto palabra a palabra.

01

Definir ventana

Se fija un tamaño de ventana (ej. $W=50$ tokens).

02

Calcular TTR local

Se calcula el TTR de cada ventana individual.

03

Deslizar y repetir

La ventana avanza un token y se repite el cálculo.

04

Promediar resultados

El MATTR final es la media de todos los TTR locales.



Resultado: Al calcular siempre sobre ventanas del mismo tamaño, el efecto de la longitud total del texto se neutraliza. MATTR es comparable entre textos de cualquier extensión.

Conexión con la Lematización

¿Qué nos dice el ratio sobre la necesidad de preprocesar el texto?

Un MATTR bajo no siempre indica pobreza real de ideas: en muchos casos refleja la proliferación de **formas flexionadas** del mismo lema. Verbos conjugados, plurales y derivados inflan artificialmente los types sin añadir semántica nueva.

Sin Lematizar

«decretó», «decretará», «decretaron», «decretos», «decreto» se cuentan como **5 types distintos**.

→ MATTR artificialmente alto o inconsistente.

Con Lematización

Todas las formas anteriores colapsan en el lema raíz **«decreto»**.

→ **1 solo type semánticamente relevante**. El MATTR refleja fielmente la diversidad conceptual real.

 **Regla práctica:** Si el MATTR es **menor de 0.40**, el texto probablemente está saturado de flexiones redundantes. La lematización se convierte en una etapa de preprocesamiento **crítica** antes de indexar en RAG.

El Impacto Crítico en RAG

Retrieval-Augmented Generation: cuando la riqueza léxica decide la calidad de la recuperación

▲ Problema 1: Embeddings dispersos

Los modelos de embeddings (Word2Vec, BERT, etc.) generan vectores distintos para cada forma flexionada. «decretó» y «decreto» producen vectores cercanos pero **no idénticos**. En la similitud de coseno, esta dispersión vectorial reduce la precisión del retrieval: documentos semánticamente equivalentes quedan más lejos en el espacio vectorial de lo que deberían.

▲ Problema 2: Chunks redundantes

Si el MATTR de un chunk es muy bajo, ese fragmento contiene poca densidad informativa. Al recuperarlo, el sistema inyecta en el **context window** del LLM texto repetitivo que:

- Ocupa tokens de contexto valiosos sin aportar información nueva
- Puede confundir al modelo con señales redundantes
- Reduce la calidad y precisión de la respuesta generada

❌ **Conclusión sistémica:** No medir la riqueza léxica antes de construir tu pipeline RAG es como calibrar un sistema de precisión con datos de baja calidad. El error se propaga a cada consulta.

Tabla de Decisión Rápida

Tu semáforo de preprocesamiento para sistemas RAG



MATTR > 0.60

Vocabulario rico y diverso

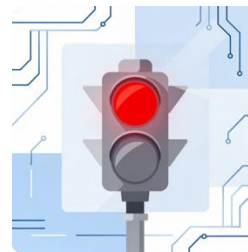
- El texto presenta alta variedad semántica
- Los embeddings serán representativos
- Los chunks aportan densidad informativa
- **Acción:** Conservar texto original sin lematizar



$0.40 \leq \text{MATTR} \leq 0.60$

Zona de evaluación contextual

- Variedad moderada, posible redundancia parcial
- Evaluar el dominio del texto (técnico vs. divulgativo)
- Lematización opcional según tipo de corpus
- **Acción:** Analizar antes de decidir



MATTR < 0.40

Burocracia / Redundancia alta

- Texto saturado de flexiones y repeticiones
- Embeddings dispersos y chunks de baja densidad
- El pipeline RAG se verá seriamente comprometido
- **Acción:** Lematización **crítica** e imprescindible

«Medir la riqueza léxica no es un lujo académico: es el primer control de calidad de cualquier sistema RAG robusto. Sin esta medición, estás indexando a ciegas.»

¿Qué es la Lematización?

Reducir la diversidad superficial para revelar la identidad semántica profunda

Definición formal

La lematización es el proceso de reducir cada forma flexionada de una palabra a su forma canónica o de diccionario, llamada **lema**. A diferencia del stemming (que recorta sufijos de forma mecánica), la lematización usa conocimiento morfológico y gramatical para obtener formas lingüísticamente válidas.

El lema siempre es una palabra real del idioma.
El stem puede no serlo.

Ejemplos en español

Verbos conjugados

«decretó → decreto»
«aprobaron → aprobar»
«implementará → implementar»

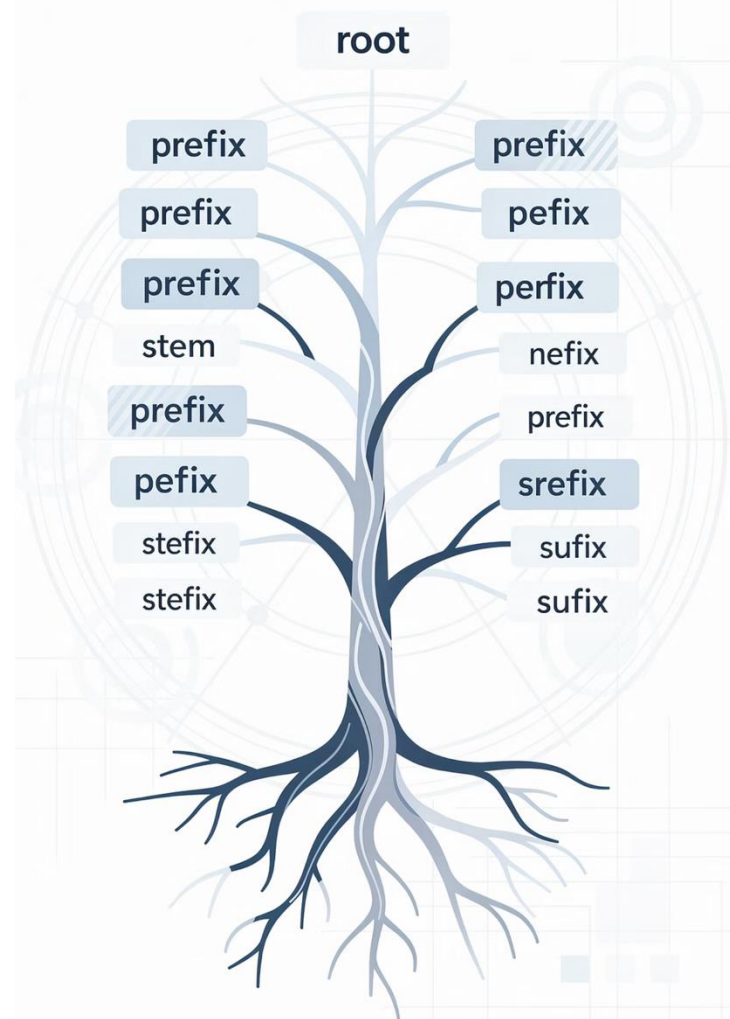
Sustantivos y plurales

«decretos → decreto»
«leyes → ley»
«presupuestos → presupuesto»

Adjetivos y derivados

«lingüísticos → lingüístico»
«semánticas → semántico»
«redundantes → redundante»

- ❗ La lematización requiere conocer la **categoría gramatical** (POS tag) de la palabra en contexto. «banco» como sustantivo → «banco»; «bancan» como verbo → «bancar». El contexto importa.



Alternativas a spaCy: NLTK y Stanza

Comparativa de librerías de lematización para elegir la herramienta correcta

spaCy ⚡

PRODUCCIÓN

VELOCIDAD

```
import spacy
nlp = spacy.load("es_core_news_sm")
doc = nlp("Los decretos aprobados")
lemas = [t.lemma_ for t in doc]
```

- 📊 Muy rápido (C++ bajo el capó)
- 📊 Modelos neuronales para español
- 📊 Ideal para pipelines en producción
- ⚠️ Modelos ocupan espacio en disco
- ⚠️ Menos flexible para investigación

NLTK 📖

INVESTIGACIÓN

CLÁSICO

```
from nltk.stem import SnowballStemmer
# NLTK hace stemming, no lematización real
stemmer = SnowballStemmer("spanish")
stem = stemmer.stem("aprobados")
# → 'aprob' ← NO es un lema válido
```

- 📊 Muy documentado y didáctico
- 📊 Excelente para aprender NLP
- 📊 Solo ofrece stemming en español
- 📊 El stem no es un lema lingüístico
- ⚠️ No recomendado para RAG en producción

Stanza 📄

PRECISIÓN

STANFORD NLP

```
import stanza
stanza.download('es')
nlp = stanza.Pipeline('es')
doc = nlp("Los decretos aprobados")
lemas = [w.lemma for s in doc.sentences
         for w in s.words]
```

- 📊 Máxima precisión lingüística
- 📊 Desarrollado por Stanford NLP
- 📊 Excelente para textos jurídicos/técnicos
- ⚠️ Más lento que spaCy
- ⚠️ Mayor consumo de memoria

Criterio	spaCy	NLTK	Stanza
Lematización real en ES	📊 Sí	📊 Solo stem	📊 Sí
Velocidad	📊 Alta	📊 Media	📊 Baja
Precisión	📊 📊 📊 📊	📊 📊	📊 📊 📊 📊 📊
Recomendado para RAG	📊 Primera opción	📊 No	📊 Si precisión crítica



Regla de oro: Usa **spaCy** como primera opción para pipelines RAG en producción. Recurre a **Stanza** cuando trabajes con textos jurídicos, médicos o científicos donde la precisión morfológica sea crítica. Evita NLTK para lematización en español.